

Laboratorio No. 04 - 2026-I - Robótica de Desarrollo, Intro a ROS 2 Jazzy Jalisco - Turtlesim



1. Requisitos:

- Tutoriales de Linux Intro Linux.
- Tutoriales de ROS 2 Intro ROS 2 Jazzy Jalisco.
- Tutoriales de Turtlesim Intro Turtlesim.
- Tutoriales de Arquitectura de funcionamiento de ROS 2 Arquitectura de Funcionamiento.

2. Resultados de aprendizaje:

- Conocer y explicar los conceptos básicos de ROS 2 (Robot Operating System 2).
- Usar los comandos fundamentales de Linux en Ubuntu 24.04.
- Conectar nodos de ROS 2 Jazzy Jalisco con Python.
- Implementar nodos propios en ROS 2 utilizando publicación, suscripción y servicios.
- Analizar la arquitectura de comunicación de una aplicación básica en ROS 2 mediante tópicos, nodos, servicios y herramientas de inspección.

3. Ejercicio en el laboratorio:

a) Familiarizarse con los comandos de mayor uso para la consola de Linux.

b) Implementación en Python:

Procedimiento:

- Dentro del **workspace** creado en clase (`my_turtle_controller`), se debe editar el archivo `move_turtle.py`. El objetivo es controlar el movimiento de la tortuga en el simulador `turtlesim` mediante el teclado, implementar trayectorias automáticas y utilizar los mecanismos básicos de comunicación de ROS 2 Jazzy Jalisco, cumpliendo con los siguientes requerimientos:

1) Control de movimiento manual:

- El script debe permitir mover la tortuga de forma **lineal y angular** utilizando las **flechas del teclado**.
- Acciones asignadas:
 - Flecha ↑: avanzar hacia adelante.

- Flecha ↓: retroceder.
- Flecha ←: girar a la izquierda.
- Flecha →: girar a la derecha.

2) Acciones complementarias y trayectorias automáticas:

- El mismo script `move_turtle.py` deberá incorporar funciones adicionales que permitan ejecutar acciones especiales y trayectorias automáticas desde el teclado.
- Las funciones deben estar implementadas de forma modular, usando métodos independientes para cada acción.
- Acciones mínimas requeridas:
 - Tecla **S**: dibujar un cuadrado.
 - Tecla **T**: dibujar un triángulo equilátero.
 - Tecla **R**: reiniciar la posición de la tortuga.
 - Tecla **P**: activar o desactivar el lápiz de dibujo.
 - Tecla **A**: ejecutar una trayectoria automática evitando que la tortuga salga de los límites de la ventana.
 - Tecla **Q**: detener completamente el movimiento de la tortuga.
- Para las trayectorias automáticas, el estudiante deberá definir velocidades lineales, velocidades angulares y tiempos de ejecución adecuados para generar las figuras solicitadas.

3) Dibujo automático de letras personalizadas:

A partir del nombre completo de cada uno de los integrantes del equipo, por ejemplo, *Manuel Felipe Carranza Montenegro*, se deben implementar funciones para que la tortuga dibuje las letras correspondientes a las iniciales de sus nombres y apellidos. En este caso específico, se deben incluir las siguientes teclas:

- Tecla **M**: dibujar la letra **M**, por *Manuel* y *Montenegro*.
- Tecla **F**: dibujar la letra **F**, por *Felipe*.
- Tecla **C**: dibujar la letra **C**, por *Carranza*.

Cada grupo deberá adaptar las letras de acuerdo con las iniciales de los integrantes. Las funciones de dibujo deben estar documentadas en el código y deben poder ejecutarse desde el teclado.

4) Sistema líder-seguidor con dos tortugas:

- Se debe crear una segunda tortuga en el simulador utilizando los servicios disponibles en `turtlesim`.
- La tortuga principal, `turtle1`, será controlada mediante el teclado y ejecutará las funciones manuales y automáticas descritas anteriormente.
- La segunda tortuga, por ejemplo `turtle2`, deberá seguir automáticamente la posición de la tortuga principal.
- Para implementar este comportamiento, el estudiante deberá:
 - Leer la posición de `turtle1` a partir del tópico correspondiente.
 - Leer la posición de `turtle2` a partir del tópico correspondiente.
 - Calcular la distancia y orientación relativa entre ambas tortugas.

- Publicar comandos de velocidad para que `turtle2` se aproxime progresivamente a `turtle1`.

- El comportamiento de seguimiento debe implementarse mediante nodos propios en Python, utilizando `rclpy`.

5) **Restricciones importantes:**

- El movimiento de la tortuga debe ser gestionado **exclusivamente desde el script** `move.turtle.py` y los nodos propios creados por el estudiante.
- **No está permitido utilizar el nodo `turtle_teleop_key`** para el control con teclado.
- Las funciones automáticas no deben bloquear permanentemente la ejecución del nodo.
- El código debe estar organizado, comentado y documentado para facilitar su revisión.

c) **Verificación de la arquitectura ROS 2:**

- Durante el desarrollo del laboratorio, se debe inspeccionar la arquitectura de comunicación generada por los nodos, tópicos y servicios utilizados.
- El repositorio deberá incluir capturas de pantalla o salidas de consola donde se evidencie el uso de los siguientes comandos:
 - `ros2 node list`
 - `ros2 topic list`
 - `ros2 topic echo /turtle1/pose`
 - `ros2 topic info /turtle1/cmd_vel`
 - `ros2 service list`
 - `rqt_graph`
- Cada comando debe ir acompañado de una breve explicación en el archivo `README.md`, indicando qué información permite observar y cómo se relaciona con la arquitectura de ROS 2.

Actividades a realizar

a) **Documentación del desarrollo:**

Se debe incluir una descripción detallada del desarrollo del laboratorio en el archivo `README.md` del repositorio de GitHub. Este documento debe explicar claramente los objetivos, procedimientos realizados, decisiones de diseño y funcionamiento general del proyecto.

La documentación debe incluir como mínimo:

- Descripción general del laboratorio.
- Explicación del control manual de la tortuga.
- Explicación de las funciones automáticas implementadas.
- Explicación del dibujo de letras personalizadas.
- Explicación del sistema líder-seguidor con dos tortugas.
- Descripción de los nodos, tópicos y servicios utilizados.
- Evidencias de ejecución del programa.

b) Diagrama de flujo:

Se debe elaborar un diagrama de flujo que represente de manera clara el funcionamiento de la solución planteada. Este diagrama debe ser implementado utilizando el lenguaje **Mermaid**, y debe incluirse en el **README.md** para facilitar su visualización directa en GitHub.

El diagrama debe representar como mínimo:

- Inicio del nodo.
- Lectura del teclado.
- Selección entre control manual, trayectoria automática, dibujo de letras o detención.
- Publicación de comandos de velocidad.
- Lectura de la posición de las tortugas.
- Cálculo del seguimiento de **turtle2** hacia **turtle1**.
- Finalización segura del programa.

c) Código fuente:

Todos los scripts y archivos de código utilizados deben ser añadidos como anexos dentro del repositorio, organizados en una estructura de carpetas clara y comprensible. Es importante incluir comentarios relevantes en el código para facilitar su comprensión.

El código debe cumplir con los siguientes criterios:

- Uso de Python con **rclpy**.
- Separación clara de funciones.
- Comentarios explicativos en las partes principales del código.
- Implementación de publicación y suscripción en ROS 2.
- Uso de servicios para la creación o reinicio de tortugas cuando corresponda.
- Manejo adecuado de errores básicos durante la ejecución.

d) Evidencia de funcionamiento:

El repositorio deberá incluir evidencia del funcionamiento del laboratorio mediante imágenes, capturas de consola o archivos de apoyo. Esta evidencia debe permitir verificar que el programa fue ejecutado correctamente en ROS 2 Jazzy Jalisco.

Como mínimo, se deben incluir evidencias de:

- Movimiento manual de la tortuga.
- Dibujo de figuras geométricas.
- Dibujo de letras personalizadas.
- Funcionamiento del sistema líder-seguidor.
- Salida de comandos de inspección de nodos, tópicos y servicios.
- Visualización de la arquitectura mediante **rqt_graph**.

e) Video explicativo:

Se debe grabar un video con una duración máxima de 10 minutos, en el que se analice, desarrolle e implemente la solución planteada. El video debe cumplir con la siguiente estructura:

- **Introducción:** el video debe comenzar con la introducción oficial del laboratorio LabSIR Intro LabSIR.

- **Presentación del equipo:** incluir una diapositiva donde se presenten los integrantes del grupo, sus nombres y aportes dentro del laboratorio.
- **Análisis, desarrollo e implementación:** mostrar el proceso seguido para resolver la actividad, incluyendo explicaciones del código, decisiones técnicas tomadas y evidencia del funcionamiento correcto de la solución.
- **Arquitectura ROS 2:** explicar brevemente los nodos, tópicos y servicios usados durante el laboratorio.
- **Conclusiones:** incluir una sección final donde se expongan las conclusiones generales del laboratorio, reflexiones del equipo y aprendizajes obtenidos.

Observaciones:

- a) Se debe crear un repositorio en GitHub donde se documente la metodología, los resultados, análisis y conclusiones.
- b) **Forma de trabajo:** grupos de laboratorio de 2 personas.
- c) Los puntos que requieran implementación deben tener funciones bien comentadas, y se deben adjuntar los archivos `.py` correspondientes.
- d) No se aceptará el uso de `turtle_teleop_key` como solución principal para el control de la tortuga.
- e) La solución debe ejecutarse en Ubuntu 24.04 con ROS 2 Jazzy Jalisco.

Entrega:



- a) **Forma de trabajo:** en parejas. Cada integrante deberá incluir la URL del repositorio.
- b) **Entregables:** repositorio en GitHub con lo solicitado en las actividades y ejercicios.
- c) **Fecha de entrega:** según la programación en Moodle.

Referencias

- [1] LabSIR UN. *Intro Linux - Ubuntu 24.04* [Repositorio en GitHub]. Disponible en: https://github.com/labsir-un/01_Rob_2026_I_Intro_Ubuntu_24.04.git
- [2] LabSIR UN. *Intro ROS 2 Jazzy Jalisco* [Repositorio en GitHub]. Disponible en: https://github.com/labsir-un/02_Rob_2026_I_Intro_ROS2_Jazzy.git
- [3] LabSIR UN. *Intro Turtlesim con ROS 2 Jazzy Jalisco* [Repositorio en GitHub]. Disponible en: https://github.com/labsir-un/03_Rob_2026_I_ROS2_Jazzy_Turtlesim.git
- [4] LabSIR UN. *Arquitectura de funcionamiento de ROS 2 Jazzy Jalisco* [Repositorio en GitHub]. Disponible en: https://github.com/labsir-un/04_Rob_2026_I_ROS2_Jazzy_Architecture.git
- [5] Open Robotics. *ROS 2 Jazzy Jalisco Installation Guide*. Disponible en: <https://docs.ros.org/en/jazzy/Installation.html>

- [6] Open Robotics. *Ubuntu Deb Packages - ROS 2 Jazzy Jalisco*. Disponible en: <https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html>
- [7] Open Robotics. *Using turtlesim, ros2, and rqt - ROS 2 Jazzy Jalisco Tutorial*. Disponible en: <https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Introducing-Turtlesim/Introducing-Turtlesim.html>